



Symphony6 NOCS3.x Chipset Specific Configuration for STBMs

Document Number:

Revision Number: 1.2

Revision Date: Jun 26, 2025

Montage LZ Technology Proprietary and Confidential

Revision History

Rev.	Date	Author	Comment
1.0	May 29, 2025	Montage-LZ	Init Version
1.1	Jun 23, 2025	Montage-LZ	Update according to review
1.2	Jun 26, 2025	Montage-LZ	Update DTE boot size
1.3	Jan 30, 2026	Montage-LZ	Add OTP fields that be set by NAGRA APIs.

1 Contents

1	Introduction.....	4
2	Flash Memory Map.....	4
2.1	SCS_Params.....	6
2.2	DTE_BOOTCODE.....	8
2.3	Boot Entry Point (Flash)	9
3	DMA-Like I/F Protection	9
4	Suspend-to-RAM(STR)	9
5	OTP Specific Configuration for STBMs.....	12
6	Configurations/Notifications for Secure Boot	15
7	Notifications for Development/Debug and Production of STBs	17
7.1	Development/Debug Phase	17
7.1.1	Debug Tool.....	17
7.1.2	HDCP Key.....	19
7.2	Production Phase	19
7.2.1	Encrypt HDCP Key	19
7.2.2	Encrypt FlashProtectionKey	19
8	Acronyms/Terms.....	19

1 Introduction

This document introduces the Symphony6 configuration for STBMs. The purpose of this document is to assist STBMs who are going to integrate Symphony6 into their NASC STB configuring flash map, external RAM, and other security-related settings.

The main contents of this document are listed below:

- Descriptions of Flash Memory Map Information
- Descriptions of External RAM Memory Map
- Notifications for Suspend-to-RAM(STR)
- Configurations/Notifications for OTP fusing in development and production phases of STBs
- Configurations for Secure Boot
- Other configurations for Development/Debug/Production of STBs

2 Flash Memory Map

With the design of Symphony6 NOCS configuration, Symphony6 supports boot from each of SPI NOR, EMMC, SPI-NAND or Parallel NAND. Figure 1 is the SCS_TOTAL_AREA Flash Map for SPI NOR and EMMC, figure 2 is the SCS_TOTAL_AREA Flash Map for SPI NAND and Parallel NAND. STBM can select the boot device by the strapping pin or OTP configuration. Please refer to the datasheet for the specific configuration.

spi nor flash map			
SIZE(Byte)	AREA	OFFSET(HEX)	value depends on
512	Reserved	0	
2048	SCS_Param	200	
256	Sig[SCS_Param]	A00	
512	SCS_Certificate	B00	
256	Sig[SCS_Certificate]	D00	
57744	SCS_AuxCode	E00	
256	Sig[SCS_AuxCode]	EF90	
256	Sig_2nd[SCS_AuxCode]	F090	
624	Non-signed_Param	F190	
512	TEE_Certificate	F400	
256	Sig[TEE_Certificate]	F600	
2048	TEE Loader Header	F700	
256	Sig[TEE Loader Header]	FF00	
65280	TEE Loader	10000	TEE_Loader_Offset in TEE_Loader_Header
256	Sig[TEE Loader]	1FF00	
8192	Unchecked Area	20000	
0x3DF00/0x7DF00/0xFDF00/0x1FDF00	DTE_BootCode	22000	SCS_DTE_BootCode_Offset in SCS_Param
256	Sig[DTE_BootCode]	5FF00/9FF00/11FF00/21FF00 60000/A0000/120000/220000	

Figure 1 SCS_TOTAL_AREA Flash Map for SPI NOR and EMMC

For SPI_Nor flash, the total SCS Area size is 0x60000, 0xA0000, 0x120000 or 0x220000 depends on different DTE_BootCode size. The 512 bytes reserved region from 0 address is to unify the OFFSET with the definition on Parallel-Nand flash, in SPI Nor-Flash case, this area is not loaded by BootROM in SCS, the data inside it can be any value.

nand flash map			
SIZE (DEC)	AREA	OFFSET (HEX)	value depends on
512	Reserved	0	
2048	SCS_Param	200	
256	Sig[SCS_Param]	A00	
512	SCS_Certificate	B00	
256	Sig[SCS_Certificate]	D00	
57744	SCS_AuxCode	E00	
256	Sig[SCS_AuxCode]	EF90	
256	Sig_2nd[SCS_AuxCode]	F090	
624	Non-signed_Param	F190	
512	TEE_Certificate	F400	
256	Sig[TEE_Certificate]	F600	
2048	TEE_Loader_Header	F700	
256	Sig[TEE_Loader_Header]	FF00	
65280	TEE_Loader	10000	TEE_Loader_Offset in TEE_Loader_Header
256	Sig[TEE_Loader]	1FF00	
393216	Reserved for bad block	20000	Just a example, can be decided from block size and DTE Boot Code size
8192	Unchecked Area	80000	
0x3DF00/0x7DF00/0xFDF00/0x1FDF00	DTE_BootCode	82000	SCS_DTE_BootCode_Offset in SCS_Param
256	Sig[DTE_BootCode]	BFF00/FFF00/17FF00/27FF00	
		C0000/100000/180000/280000	

Figure 2 SCS_TOTAL_AREA Flash Map for SPI NAND

The difference in SPI-Nand flash map from SPI-Nor is just the additional area behind Sig[TEE_Loader] reserved for bad block management, its size is defined as 384K bytes.

Also, the 512 bytes reserved region from 0 address is to unify the OFFSET with the definition on Parallel-Nand flash, in SPI-Nand Flash case, this area is not loaded by BootROM in SCS, the data inside it can be any value.

nand flash map			
SIZE (DEC)	AREA	OFFSET (HEX)	value depends on
512	Reserved	0	
2048	SCS_Param	200	
256	Sig[SCS_Param]	A00	
512	SCS_Certificate	B00	
256	Sig[SCS_Certificate]	D00	
57744	SCS_AuxCode	E00	
256	Sig[SCS_AuxCode]	EF90	
256	Sig_2nd[SCS_AuxCode]	F090	
624	Non-signed_Param	F190	
512	TEE_Certificate	F400	
256	Sig[TEE_Certificate]	F600	
2048	TEE_Loader_Header	F700	
256	Sig[TEE_Loader_Header]	FF00	
65280	TEE_Loader	10000	TEE_Loader_Offset in TEE_Loader_Header
256	Sig[TEE_Loader]	1FF00	
393216	Reserved for bad block	20000	Just a example, can be decided from block size and DTE Boot Code size
8192	Unchecked Area	80000	
0x3DF00/0x7DF00/0xFDF00/0x1FDF00	DTE_BootCode	82000	SCS_DTE_BootCode_Offset in SCS_Param
256	Sig[DTE_BootCode]	BFF00/FFF00/17FF00/27FF00	
		C0000/100000/180000/280000	

Figure 3 SCS_TOTAL_AREA Flash Map for Parallel Nand

The difference in Parallel-Nand flash map from SPI-Nand is just the 512 bytes from 0 address, in SPI-Nand case, the original device configuration information is stored in the first page by the Nand manufactory, it is used by BootROM in the power on reset to identify which kind of Nand type the on-board device is, so that is why we put the SCS_Param to the 521 bytes offset all for SPI-Nor, SPI-Nand and Parallel-Nand.

In the figures listed above, there are 4 Nagra NOCS blocks, and 3 TEE blocks:

- SCS_Params
- SCS_Certificate
- SCS_AuxCode
- DTE_BootCode

Offsets of all Nagra NOCS blocks within SCS_TOTAL_AREA are fixed. Please check table below for details.

Table 1 Nagra NOCS blocks within SCS_TOTAL_AREA

Name of NOCS block	Address	Size
SCS_Params+Signature	0x200	0x900
SCS_Certificate+Signature	0xB00	0x300
SCS_AuxCode+Signature+Signature2	0xE00	0xE390
DTE_BootCode+Signature	Fixed in SCS_Params, named SCS_DTE_BootCode_Offset , for SPI Nor-Flash, it is 0x22000, for SPI-Nand-Flash and Parallel-Nand , it is 0x82000	0x3E000/0x7E000/0xFE000/0x1FE000

In all these NOCS blocks, the configurable parameters for STBM are located in SCS_Params_. Please refer the section below for details.

Table 2 TEE blocks within SCS_TOTAL_AREA

Name of NOCS block	Address	Size
TEE_Certificate+Signature	0xF400	0x300
TEE Loader Header+ Signature	0xF700	0x900
TEE Loader+ Signature	0x10000	0x10000

All TEE blocks are controlled and released by Montage-LZ. STBMs do not need to know the detail about them.

2.1 SCS_Params

Figure 4 is the memory map of SCS_Params.

Name	Size	Comment
Init_Flag	4 Bytes	Magic of flash image
Scrambling_Activation	4 Bytes	Enable the DRAM scrambling
Scrambling_Mode	4 Bytes	DRAM scrambling mode, available only when Scrambling_Activation is enabled. 0x00000000: re-new the scrambling key without key preservation. 0x55555555: re-new the scrambling key
Default_Area_Min_Size	4 Bytes	Minimal size of default area.
SCS_DTE_BootCode_Offset	4 Bytes	Flash offset of DTE_BootCode
SCS_DTE_BootCode_LoadAddr	4 Bytes	DRAM loading address of DTE_BootCode
SCS_Total_Area_Size	4 Bytes	Total size of flash image.(0x4F400/0x8F400/0x10F400/0x20F400)
DTEEnc_AfterSign_Ena (Flash_Protection_Activation)	4 Bytes	The encryption for DTE Boot Code is enabled in the case of signing on plaintext.
DTEEnc_AfterSign_Ena (Flash_Protection_Activation)	4 Bytes	The encryption for DTE Boot Code is enabled in the case of signing on plaintext.

Reserved	32 Bytes	
EDCK (FlashProtectionKey)	16 Bytes	The encrypted DTE_BootCode encryption key, in Nagra case, it is same as FlashProtectionKey.
AVCPU_Ena	4 Bytes	The flag to indicate if configure firewall in SCS for AVCPU start up. 0x0: AVCPU is disabled; others: AVCPU is enabled.
Reserved	4 Bytes	
Ext_RAM_Params	512 Bytes	The parameters for external RAM initialization
Firewall_Config	1432 Bytes	Firewall configuration after SCS finished(including the 32 bytes HMAC-SHA256 signature at the end)
MSID_Ext	4 Bytes	
MSID_Mask	4 Bytes	
VER_Ext	4 Bytes	
VER_Mask	4 Bytes	
Signature	256 Bytes	Signature of SCS_Param. Signed by CA_RSA_Ext

Figure 4 SCS_Params

In SCS_Params, a set of fields should be set by STBM. Below is the detail description of fields which must be set in SCS_Params.

- DTEnc_AfterSign_Ena (Flash_Protection_Activation)

The flash protection activation flag is provided to notify if DTE_BootCode is encrypted. Please check table below for setting of this flag.

Name	Value	State
Flash_Protection_Activation	0xAAAA5555	DTE_BootCode is encrypted
Flash_Protection_Activation	0x00000000(Other values are forbidden)	DTE_BootCode is plaintext

- EDCK [FlashProtectionKey]_{FPK-ProtKey}

It is the encrypted format of FlashProtectionKey. The FlashProtectionKey is used to encrypt/decrypt DTE_BootCode. The FlashProtectionKey is managed by STBM, and protected by the FPK-ProtKey. FPK-ProtKey is derived from FlashProt-RootKey which is provided by chipset.

NOTE: In NOCS3.0 chipset, this field is always decrypted during SCS procedure.

- SCS_DTE_BootCode_LoadAddr

It is the external RAM address where the SCS_DTE_BootCode is loaded into it.

- SCS_DTE_BootCode_Offset

It is the flash offset address where the SCS_DTE_BootCode is located.

- SCS_Total_Area_Size

It is the SCS total area size which includes the 4 Nagra NOCS blocks and 3 TEE blocks described above. **The valid option sizes are 0x4F400/0x8F400/0x10F400/0x20F400.**

- MSID_EXT & MSID_Mask

These 2 fields are used to validate Market Segment ID in SCS_Params. They will compare with the corresponding fields in OTP at SCS phase. If compare result is failed, chipset will reset.

- VER_EXT & VER_Mask

These 2 fields are used to validate Version in SCS_Params. They will compare with the corresponding fields in OTP at SCS phase. If compare result is failed, chipset will reset.

The version incrementing mechanism is described below.

The version is determined by counting the number of 1s, so the coding format is not restricted. However we recommend the STBMs to apply a coding format described in following table for convenience.

Table 2 Example of Version Format

Version	Version_EXT
1	00000001b
2	00000011b
3	00000111b
4	00001111b

The default version in OTP and SCS_Params are 0.

2.2 DTE_BOOTCODE

According to the definition above, the DTE_BOOTCODE is defined below.

As shown in Figure 5, the DTE_BootCode contains a header which combined with [Image] is stored after being encrypted by the FlashProtectionKey. The header contains the fields as described below.

- 1) Block_ID: Used to identify the NOCS Trusted Data Block.
- 1) Entry_Addr: Indicate the entry address of the DTE_BootCode.
- 2) Code_Size: Indicate the size of the text segment in the DTE_BootCode.
- 3) Body_Size: Indicate the size of the DTE_BootCode.

The size and address of DTE_BootCode is specified in the SCS_Params. The Reserved area above in this table will be totally programmed to the value 0x0.

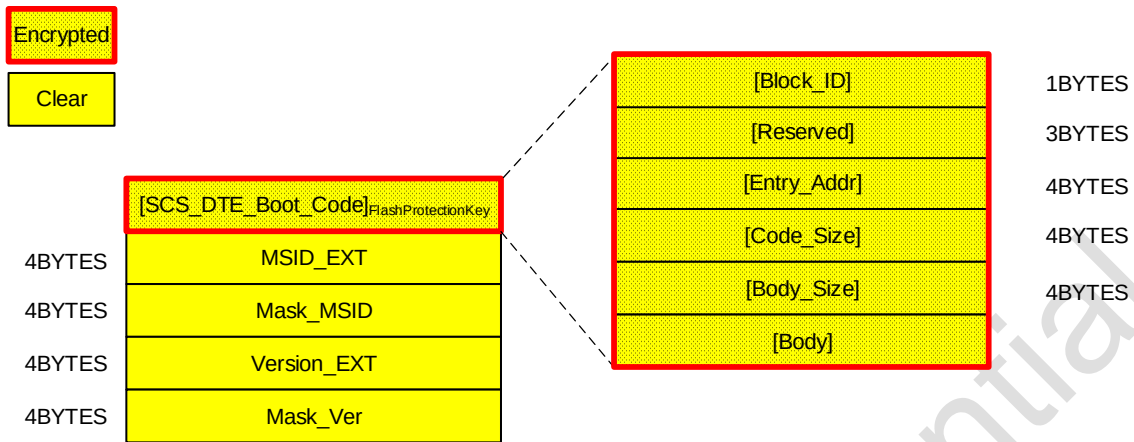


Figure 5 DTE_BootCode Memory Map

Definitions of MSID_EXT, Mask_MSID, Version_EXT, Mask_Ver are same with fields in SCS_PARAMS.

2.3 Boot Entry Point (Flash)

The Boot Entry Point (BEP) is the start address of [Image] in Figure 5. Based on Figure 4, BEP is SCS_DTE_BootCode_Offset + 16 Bytes.

For SPI NOR:

$$BEP = 0x22010$$

For SPI NAND and Parallel Nand:

$$BEP = 0x82010$$

3 DMA-Like I/F Protection

On Symphony6 there are no DMA-like interface opened for STBM, and so there are no actions needed by the STBM to protect these interface.

4 Suspend-to-RAM(STR)

STR is an optional mechanism provided by Symphony6. It is controlled by a field called OTP_STRFunc in OTP. STBM can call xxx_otp_write() to disable this function. The default value is open. There two set of data in AO module. One is "Operational Data Area", which can be accessed by APCPU. The other is "Sensitive Data", which can be accessed by pure HW only.

Figure 6 is the main frame of STR. AO is short for Always-On components in Symphony6.

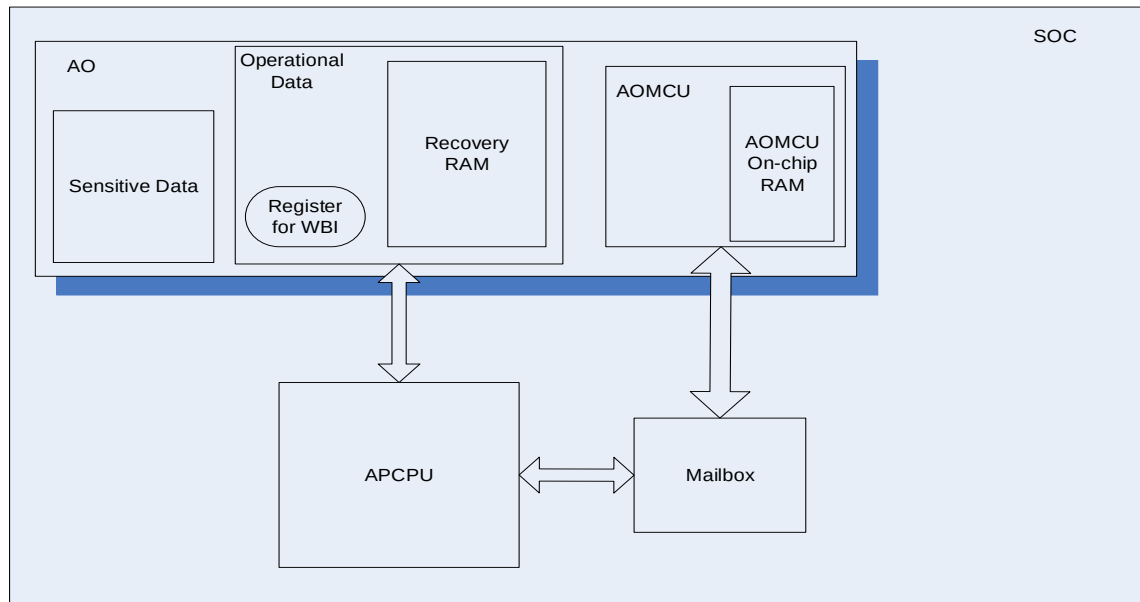


Figure 6 Resources used by Software provided by AO

STR has 2 stages, namely Suspend-to-RAM and Resume-from-RAM. OEM doesn't need to care about the details of two stages, so we only introduce the APIs that OEM needs to call.

There are several parameters need to be set/noticed by STBM.

- Default_Area_Min_Size

This size is fixed to 64K bytes. It is the minimal size of "Default_Area". If the size of default area set by STBM is smaller than this value, the chipset reboots.

It is defined in SCS_PARAMS, its format is defined below.

Table 4 Format of Default_Area_Minimal_Size

Name	Value(byte)	Description
Default_Area_Min_Size	64k	Minimum size of Default_Area to be authenticated

- Default_Area

Default_Area is the authentication area when resuming from STR. The CEP (return address of STR) must be included in this default area. If CEP is outside of Default_Area, the chipset reboots.

Default_Area is defined by Default_Area_Address and Default_Area_Size. STBM can call API below to set these 2 parameters (check_addr and check_end).

```
invoke_fn(OPTEE_SMC_SIP_SET_PM_INFO, check_addr, check_end, 0, 0, 0, 0,
0, &res);;
```

NOTE: It is a command of function "invoke_fn" now, and we recommend the area is kernel area. You can use function "pm_get_kernel_region" to get start address and end address (check_addr and check_end).

- CEP

CEP is short for Context Entry Point. It is the return address of STR. If WBC is presented, CEP is defined as the start address of WBC.

STBM can call API below to set CEP.

```
mt_str_set_cep(CEP);
```

CEP must be included in the Default_Area. If CEP is outside of Default_Area, the chipset reboots.

NOTE: The API is in TEE, and CEP is set by TEE itself in STR flow. So it is not available for STBM.

After setting all parameters above, the memory map of default area is shown as figure 7.

Figure 8 shows main flow of “Enter into STR”.

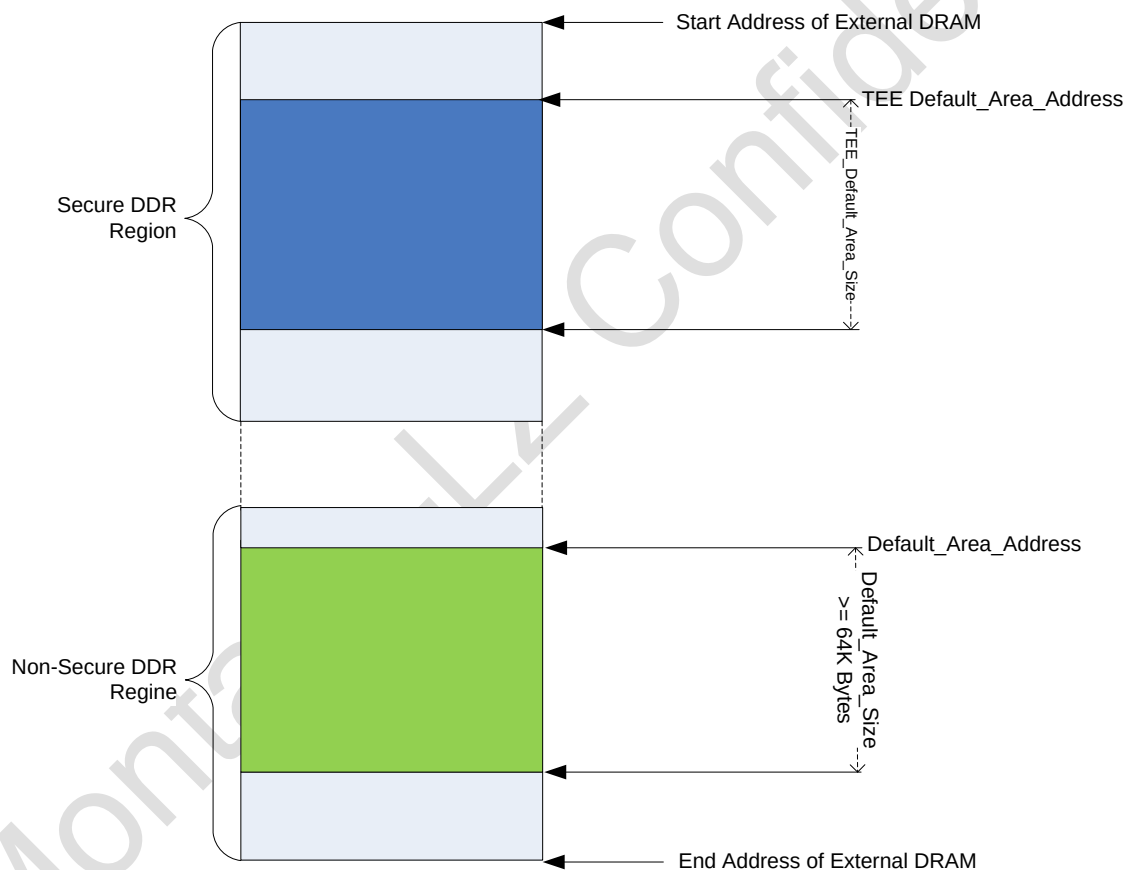


Figure 7 Example memory map in “Enter into STR”

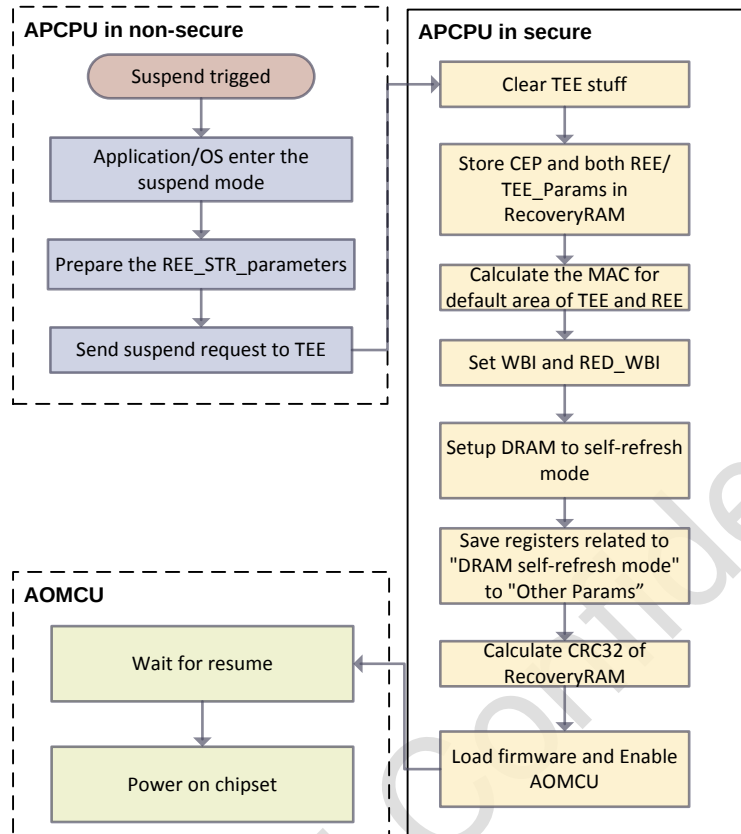


Figure 8 Enter into STR

- Enter the STR

OEM can trigger the board to STR by below command in linux.

```
system("echo mem >/sys/power/state");
```

5 OTP Specific Configuration for STBMs

To achieve security purpose, specific configurations in OTP need be set by STBMs while integrating Symphony6 into their devices.

Here is the list and its descriptions.

1), Table 5 OTP Specific Configurations for STBMs

Name	Bit Offset	Bit Length		Stage2 Value	Description
OTP_UART0_RX_Dis	0x1D3FE	2		2b11	Disable UART0 Rx
OTP_UART0_TX_Dis	0x1D3FC	2		2b11	Disable UART0 Tx
OTP_SWL_UART0	0x1ED3A	3		3b111 (locked)	Write lock for UART0

OTP_UART1_RX_Dis	0x1D3FA	2		2b11	Disable UART1 Rx
OTP_UART1_TX_Dis	0x1D3F8	2		2b11	Disable UART1 Tx
OTP_SWL_UART1	0x1ED37	3		3b111 (locked)	Write lock for UART1
OTP_STRFunc	0x1D234	4		[optional] 4b1101	Disable STR function, default is enabled. See NOTE 2
OTP_SWL_STRFunc	0x1EB4E	3		3b111	Write lock for STRFunc
OTP_APPIImageEnc_Ena	0x1F5C2	2		2b11	APP image is encrypted flag
OTP_SWL_APPIImageEnc	0x1ED68	3		3b111	Write lock for OTP_APPIImageEnc_Ena
OTP_CA_TA_MSID	0x1CFE0	32		Nagra Define	Nagra TA MSID
OTP_SWL_CA_TA_MSID	0x1EB42	3		3b111	Write lock for Nagra TA MSID
OTP_NextGuard_TA_MSID	0x1CF60	32		Nagra Define	NextGuard TA MSID
OTP_SWL_NGTA_MSID	0x1ED22	3		3b111	Write lock for NextGuard TA MSID
OTP_TEE_MSID	0x1E960	32		Montage Define	Montage-LZ private TA and TEE MSID
OTP_SWL_TEE_MSID	0x1EB65	3		3b111	Write lock for OTP_TEE_MSID
OTP_Other_TA_MSID	0x1D800	32		Third party Define	The MSID of TAs are not published by Nagra, NextGuard or Montage
OTP_SWL_CAVReserv ed15	0x1ED0E	3		3b111	Write lock for OTP_Other_TA_MSID
OTP_PNandPageSize	0x1F5F4	3		Third party Define	3'b000:strap pin define the pagesize. (0 cannot be used in Nagra case) 3'b001: page size is 512B 3'b010:page size is 2K 3'b011:page size is 4k others:page size is 8k
OTP_SPINandPageSize	0x1F5EB	2			2'b00: page size

					depend on strap pin(0 cannot be used in Nagra case) 2'b01: page size is 2KB 2'b10: page size is 4KB 2'b11: page size is 8KB
OTP_SWL_NANDBoot	0x1ED45	3		3b111	Write lock for OTP_PNandPageSize and OTP_SPINandPageSize

NOTE:

- 1) Disable UART Tx/Rx is project dependant. For example, If NASC has explicit requirement to disable UART, please follow the above table to fuse these fields.
- 2) STR function is project dependant. It is enabled by default. If STR is required, write OTP_SWL_STRRunC at Stage2 only; If STR is forbidden, write OTP_STRFunc and then write OTP_SWL_STRRunC at Stage2.
- 3) OTP_APPImageEnc_Ena is a flag. STBMs can use it to flag if APP image is encrypted.
- 4) Application should call API: MT_UNF_OTP_write () to fuse fields above. **But please note the address of API is byte offset, and table show bit offset. (byte offset = bit offset / 8).**
- 5) NAGRA request to use the OTP to select the NAND page size, so **OTP_PNandPageSize or OTP_SPINandPageSize cannot be set to 3b000 in Nagra case, please set them to other value according to your flash information.**

Besides above OTP fields, some extra features of chipset listed below are left in open status.

- USB0
- USB1
- Ethernet

2), STBM can use them based on the requirements of STB project. If some of these features need to be closed, STBM can follow table below to fuse them.

Name	Bit Offset	Bit Length	Stage2 Value	Description
USB0	0x1D51C	1	Third party Define	USB0 disable
USB1	0x1D51B	1	Third party Define	USB1 disable
OTP_EMAC_Dis	0x1D50F	1	Third party Define	Ethernet disable
OTP_SWL_Bonding	0x1ECD A	3	Third party Define	Lock USB0,1,Ethernet. Should be fused after fusing USB0,1,Ethernet.

3), In addition to the above OTP fields, there are also some other OTP fields that need to be set. However, NAGRA has defined corresponding APIs for setting these values. If you are certain that you do not want to use NAGRA's APIs to set values, you can also use our OTP operation API (MT_UNF_OTP_write and MT_UNF_OTP_read). But please note the address of API is byte offset, and table shows bit offset (byte offset = bit offset / 8).

Name	Bit Offset	Bit Length	Stage2 Value	Description
OTP_STB_CA_SN	0x1F3A0	32	NAGRA Define	CA SN value
OTP_SWL_STB_CA_SN	0x1ED02	3	3b111	After setting CA SN, should set to 3b111
OTP_CA_MSID	0x1CFA0	32	NAGRA Define	NAGRA REE market segment ID
OTP_SWL_CA_MSID	0x1ED42	3	3b111	After setting CA MSID, should set to 3b111
OTP_SECURE_EN	0x1D1E4	12	12hFFF	SCS secure boot enable
OTP_PrivilegedModeAct	0x1F3CA	4	STBM define 4b0000/4b1111	Tee privileged mode of CERT
OTP_I2CMode	0x1F7C0	9	9b00011111	Debug I2C Password protected mode
OTP_REEJTAGMode	0x1F7D2		9b00011111	JTAG Password protected mode

6 Configurations/Notifications for Secure Boot

STBMs should call NOCS API to enable Secure Boot.

Once Secure Boot is enabled, a set of elements in NOCS Trusted Data Blocks will be authenticated/checked. In NOCS3.1, Trusted Data Blocks are SCS_Params, SCS_Certificate, SCS_AuxCode, DTE_BootCode (Refer to Figure 2 for details). The elements to be authenticated/checked in these blocks are:

- SIG[SCS_xxx]
Signature of a Trusted Data Block.
All NOCS Trusted Data Blocks shall be signed by Nagra.
- MSID_EXT
External Market Segment ID of a Trusted Data Block.
- Ver_EXT

External Version of a Trusted Data Block.

According to the design, secure boot flow follows the chart as shown in Figure 9 and Figure 10.

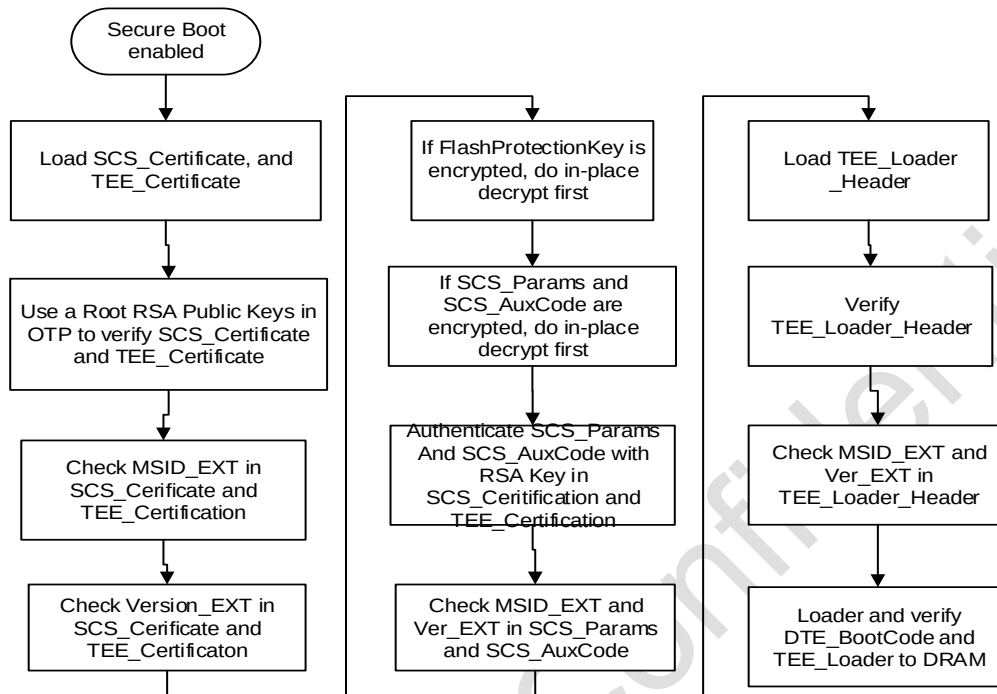


Figure 9 Secure Boot(part1)

In figure above, if any one of the steps is failed, the chipset resets. After all steps are authenticated/checked successfully, run SCS_AuxCode. The main functionalities of SCS_AuxCode are DRAM calibration, enabling DRAM scrambling, and handling STR procedure if enabled. SCS_AuxCode is developed, managed by chipset manufacturer. Then go to DTE_BootCode authentication procedure, please check Figure 10 for details. If any one of the steps is failed, the chipset resets.

About loading TEE_loader, The copy address from flash and load address in DRAM are defined by TEE_loader_Header, and it is controlled by Montage-LZ.

About Loading DTE_BootCode The copy address from flash and load address in DRAM are defined by SCS_DTE_BootCode_Offset and SCS_DTE_BootCode_LoadAddr, please check [section 2.1](#) for more details.

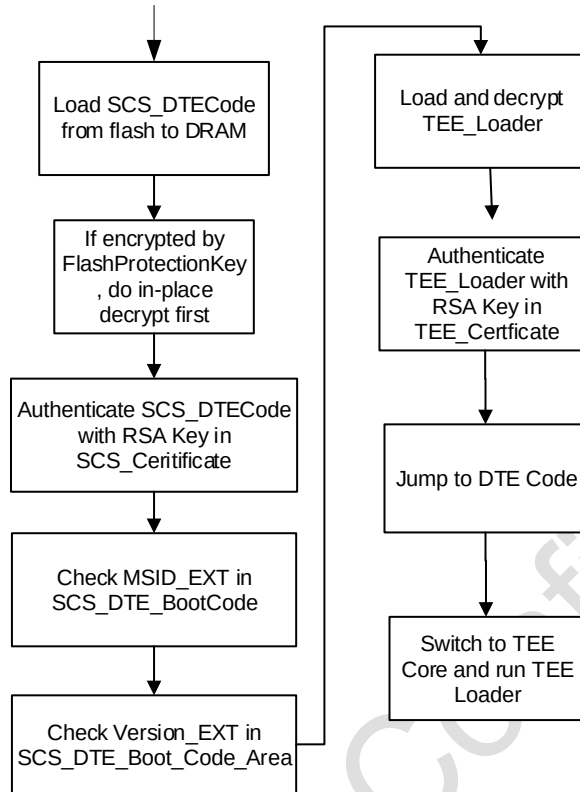


Figure 10 Secure Boot(part2)

After authentication and checking are done, clean up/mask all necessary sensitive assets in SCS procedure, then read out the Entry_Addr(defined in Figure 4), and jump to Entry_Addr to start running DTE_BootCode.

DTE_BootCode is the first boot of STBMs. Succeeding procedures are developed/controlled by STBMs based on the NASC requirements.

7 Notifications for Development/Debug and Production of STBs

Conventionally, developing an STB is separated into phases below.

- Development/Debug phase
- Production phase

7.1 Development/Debug Phase

In development/debug phase, STBMs will use tools delivered by CM to achieve boot images in accord with the flash map described above. Please refer the related manuals of package tool for how to generate boot images.

7.1.1 Debug Tool

For debug purpose, CM provides Debug I2C and JTAG tools. Based on the design, there are some notifications for STBMs before using these tools.

Descriptions of debugging are separated into 2 cases.

- Secure Boot Failed

- Secure Boot Success

7.1.1.1 Case of Secure Boot Failed

In case of secure boot failed, STBMs have to use debug tool provided by CM to connect to debug. Registers below will be set by debug tool automatically. Please refer to the manuals of tools for how to use.

Register Name	Address	Set Value	Description
Active_Debug(Write)	0x1f30d09c catch_fail	0x7	Activate debug function, or just use command "catch_fail"
Debug_State(Read)	0x1f30d09c	0x7	Debug function is activated
Set I2C password	0x1F30d010 0x1F30d014 0x1F30d018 0x1F30d01C	(Per chipset depends)	Registers for input password
Set JTAG password	0x1f30d060 0x1f30d064 0x1f30d068 0x1f30d06C	The same with I2C password	Registers for input password

To get the password of target chipset, STBMs should read out NUID through Debug I2C

Register Name	Address	Value	Description
NUIDL	0x1F313EAC	(Per chipset depends)	Low 32bits of NUID
NUIDH	0x1F313EB0	(Per chipset depends)	High 32bits of NUID

7.1.1.2 Case of Secure Boot Success

In case of secure boot success, if STBMs want to connect debug tool, they should follow steps below.

- Debug I2C & JTAG in open mode
Debug I2C tool can connect directly.
To connect JTAG, call API in SDK to set pinmux:
`mt_unf_jtag_toggle();`
then connect.
- Debug I2C & JTAG in password mode
In this case, STBMs should call NOCS API to get NUID.
When target password is obtained, call API in SDK to unlock JTAG:

```
mt_unf_jtag_set_password(password, password_length);
```

All settings above are also set through Debug I2C tool directly. Please refer the manuals of tools for more details.

7.1.2 HDCP Key

HDCP Key is resided in Flash. It is loaded by STBM's APP. For debugging, STBMs should load the key by Montage API and control HDMI module in their application. Please refer to "LZ_Linux_HDMI_Userguide.pdf" for more about HDMI. About how to protect HDCP Key, please refer to the section [Production Phase](#).

7.2 Production Phase

In STB production phase, before save HDCP key, STBMs should encrypt the plaintext of HDCP key. And re-program DTE-Boot which is from NAGRA and encrypted by the FlashProtectionKey.

7.2.1 Encrypt HDCP Key

CM provides API to encrypt plaintext of HDCP1.4 key:

```
mt_unf_encrypt_hdcpkey(plaintext_key_in, encrypted_key_out, key_size);
```

7.2.2 Encrypt FlashProtectionKey

FlashProtectionKey is owned by NAGRA, used to protect DTE_BootCode, and any other data/code owned by STBMs. When STBM got the signed SCS image from NAGRA, the DTE-Boot will be encrypted by this key. Chipset provides a root protection key in OTP to protect it. FlashProtectionKey is in SCS_Params, and signed with SCS_Params in plaintext format. After signed, FlashProtectionKey shall be encrypted, and write back into SCS_Params. STBMs shall call NOCS API to encrypt it.

8 Acronyms/Terms

Name	Description
SOC	System On Chip
SCS	Secure Chipset Startup
STR	Suspend To RAM
AO	Always On module on the chipset
AOMCU	MCU on the AO, operate shutdown or wake-up of the SOC
AOFW	Firmware of the AOMCU
WBC	Warm Boot Code. Operate some pre-initializations of warm-boot
CEP	Context Entry Point
Mailbox	Hardware mechanism to allow communication between AOMCU and Host CPU
OTP_STRFunc	STR function on/off flag field in OTP

CM	Chipset Manufacturer
STBM	Set-Top-Box Manufacturer

Montage-LZ Confidential